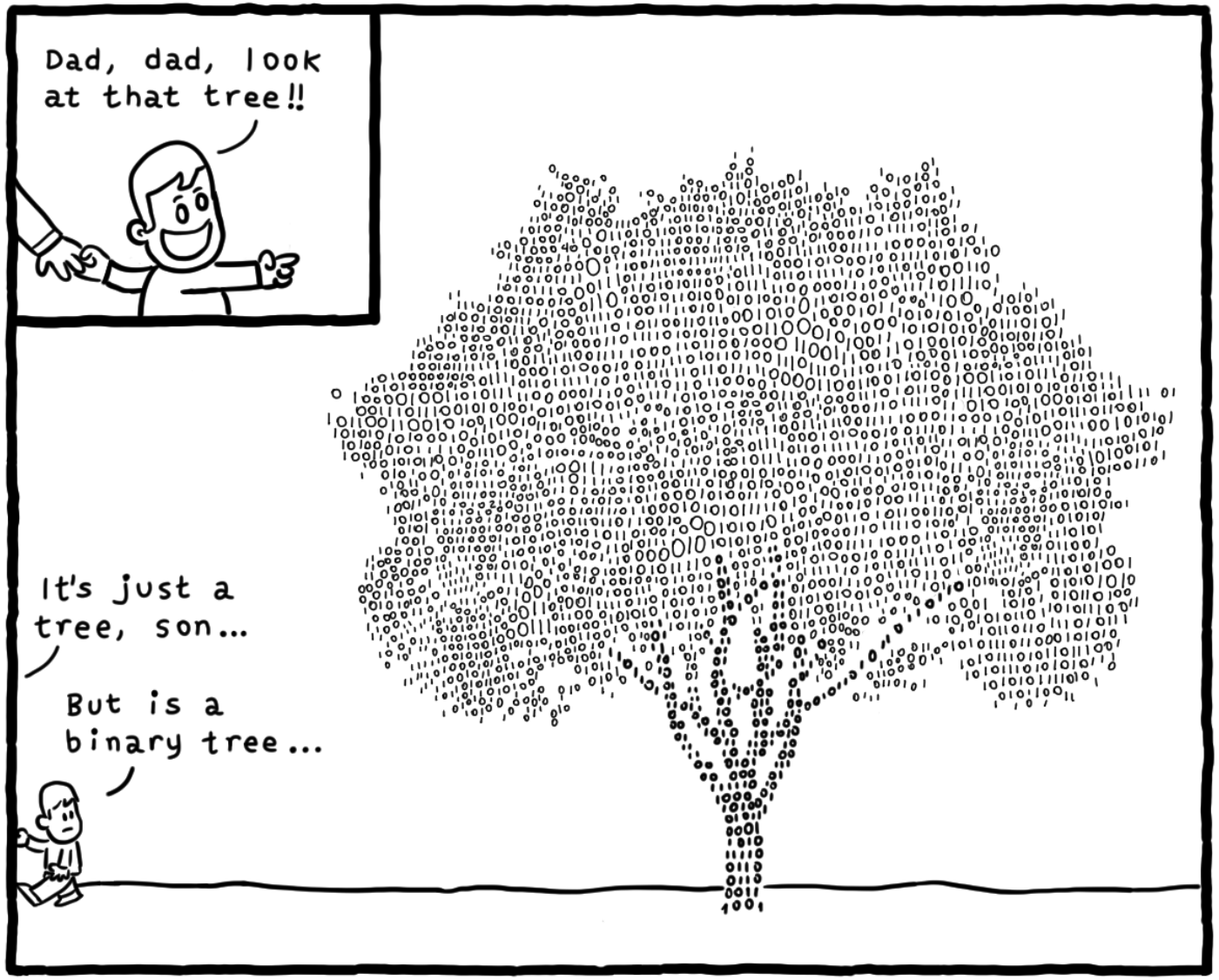
[CSED233] Data Structure

# Binary Search Tree

Seung-Hwan Baek

POSTECH




Daniel Stori {turnoff.us}

# Midterm

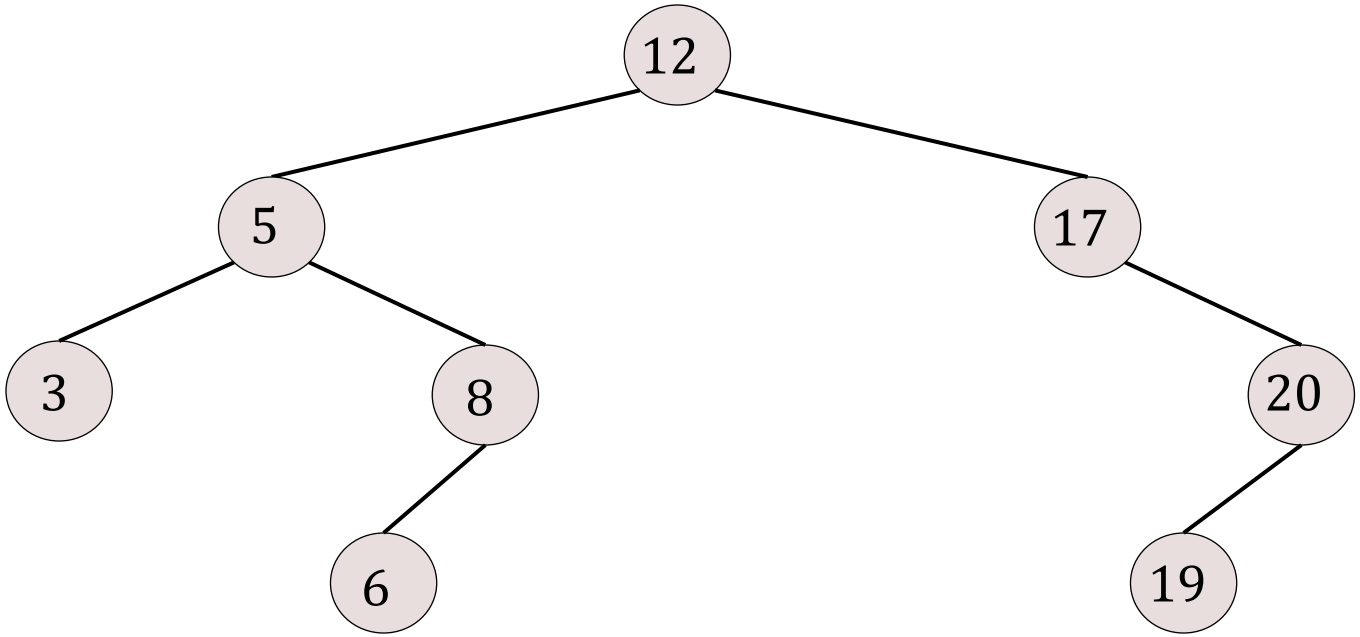
- Midterm Schedule
- Date: April 14 9:30 AM – 10:45 AM
- Location: Engineering Building 2, Room 102
- Make sure to arrive at least 5 minutes early and bring your student ID card
- Topics for midterm are the things that we will study up to midterm

# Binary Search Tree (BST)

- A binary tree
  - Each node has a $(key, value)$ pair, where $key$ is unique
- BST property
  - Every node is ordered by key which belongs to a total order
    - For any two non-equal keys $X$ & $Y$, either $X < Y$ or $X > Y$
  - The key of any node is greater than all keys stored in its left subtree & less than all keys in its right subtree

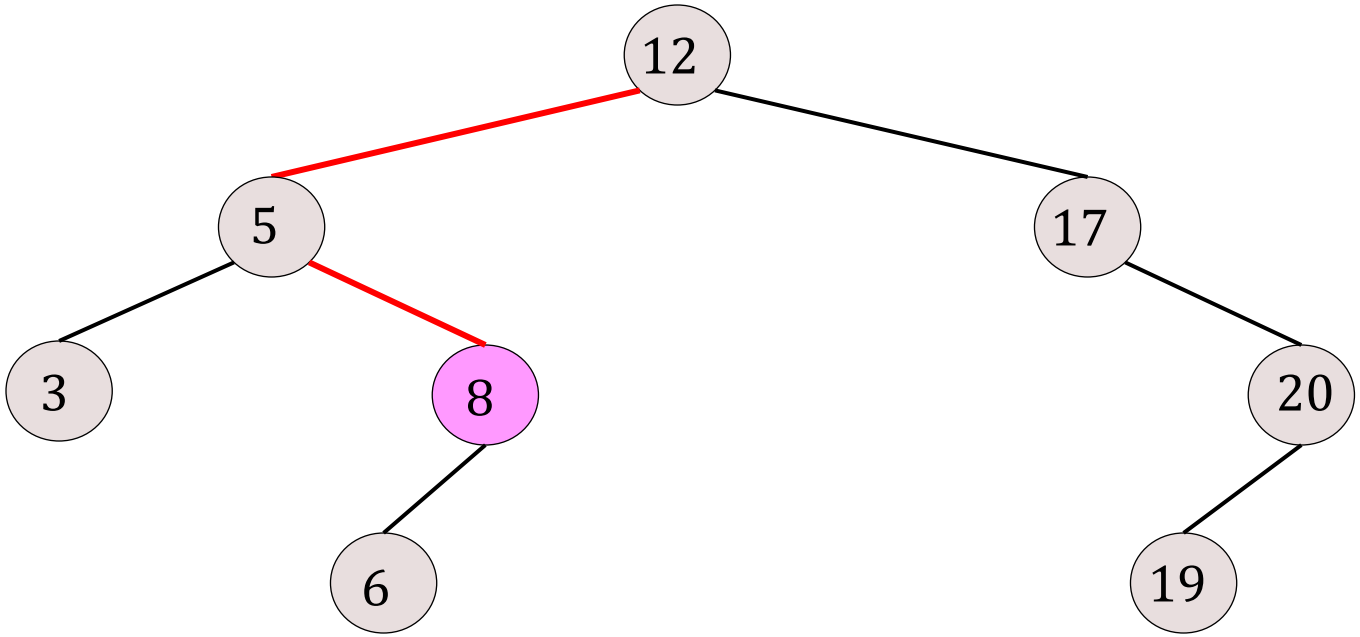
- Also known as ordered/sorted binary tree

# Binary Search Tree (BST): Example



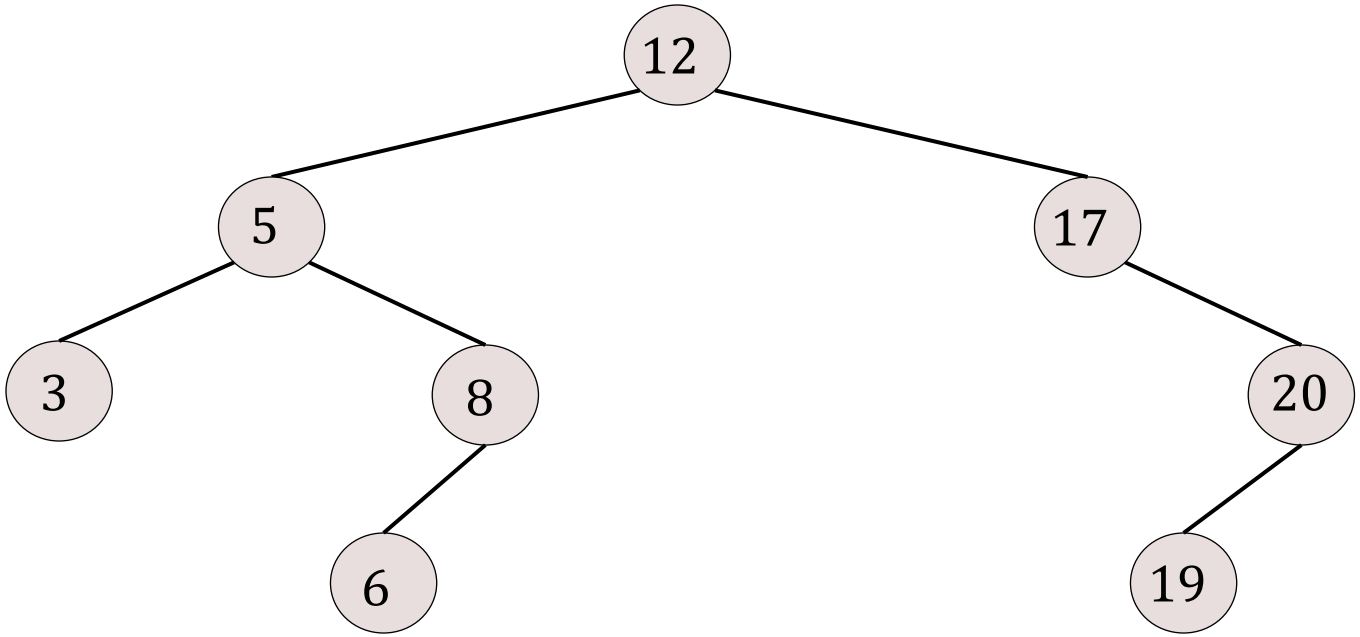

- Only keys are shown
- $Search(8, D)$

# BST: $Search(x, D)$




- After $Search(8, D)$
- Time complexity = $O(height)$
  - $O(\log n)$ if BST is balanced
  - $O(n)$ if BST is a linear list (worst case)

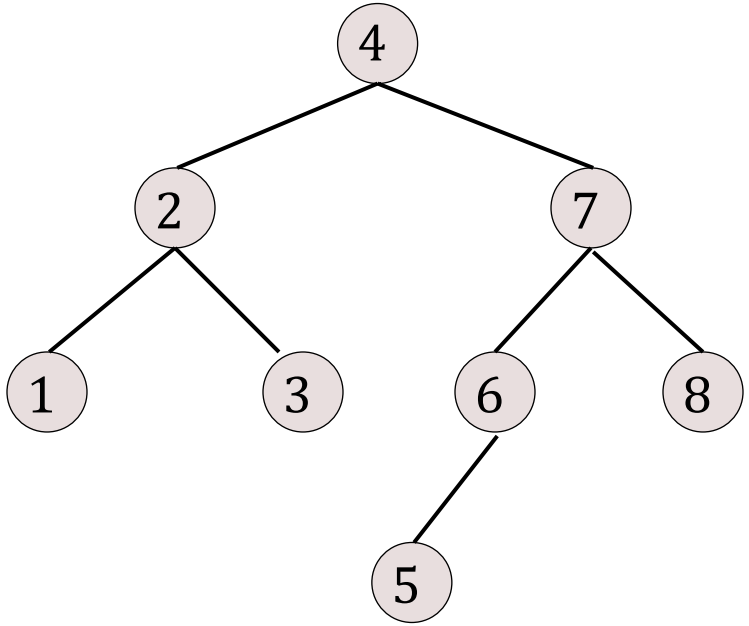
# Quiz: $Sort(D)$ in Ascending Order




- How to sort the keys in an ascending order?
- Time complexity for that?

# Inorder Tree Traversal: Binary Search Tree

- $Inorder(T) = 1, 2, 3, 4, 5, 6, 7, 8$



# Quiz

- You are given only the inorder traversal of a BST.
- Can you reconstruct the original tree structure?

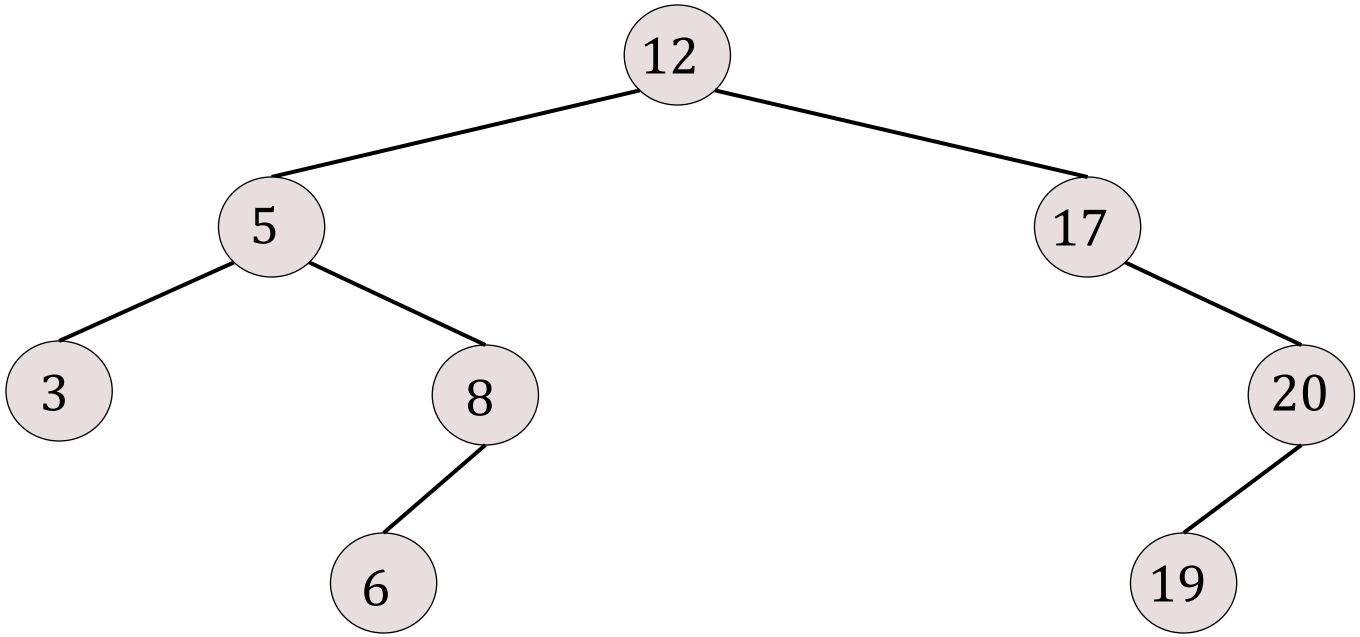
# Quiz

- You are given only the inorder traversal of a BST.
- Can you reconstruct the original tree structure?
- What do you know need to know in addition to inorder traversal?

# Quiz

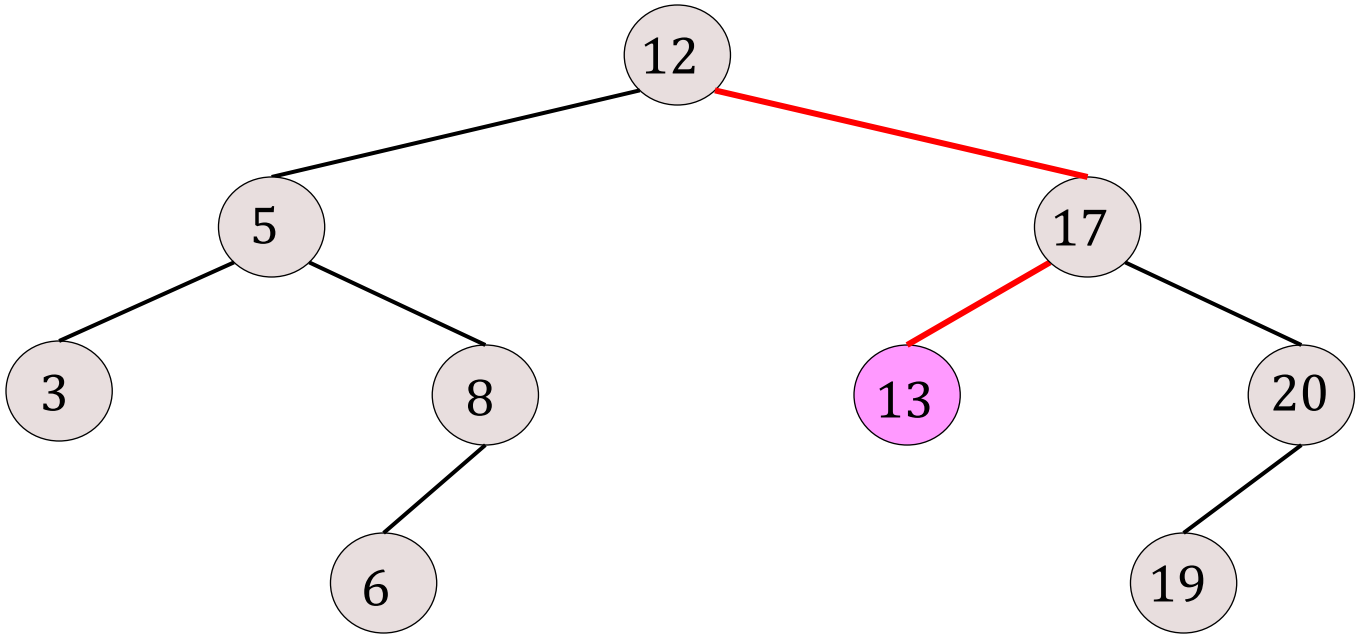
- Suppose you are given the inorder traversal and preorder traversal of a BST with distinct keys.
- Is the reconstruction algorithm fundamentally dependent on the BST p roperty, or would it also work for an arbitrary binary tree? Explain.

# BST: $Insert(x, D)$



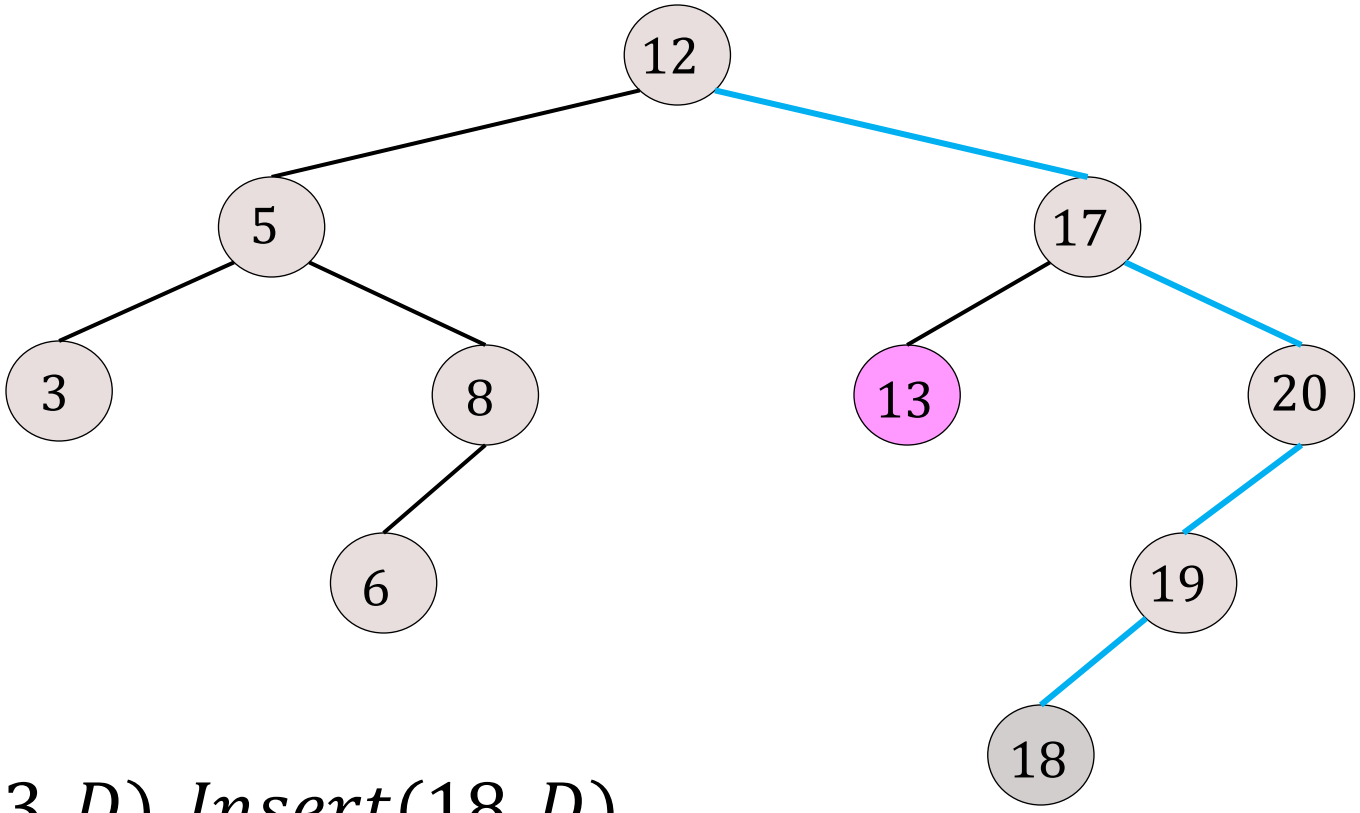

- $Insert(13, D)$

# BST: $Insert(x, D)$




- After $Insert(13, D)$
- $Insert(18, D)$

# Quiz: $Insert(x, D)$

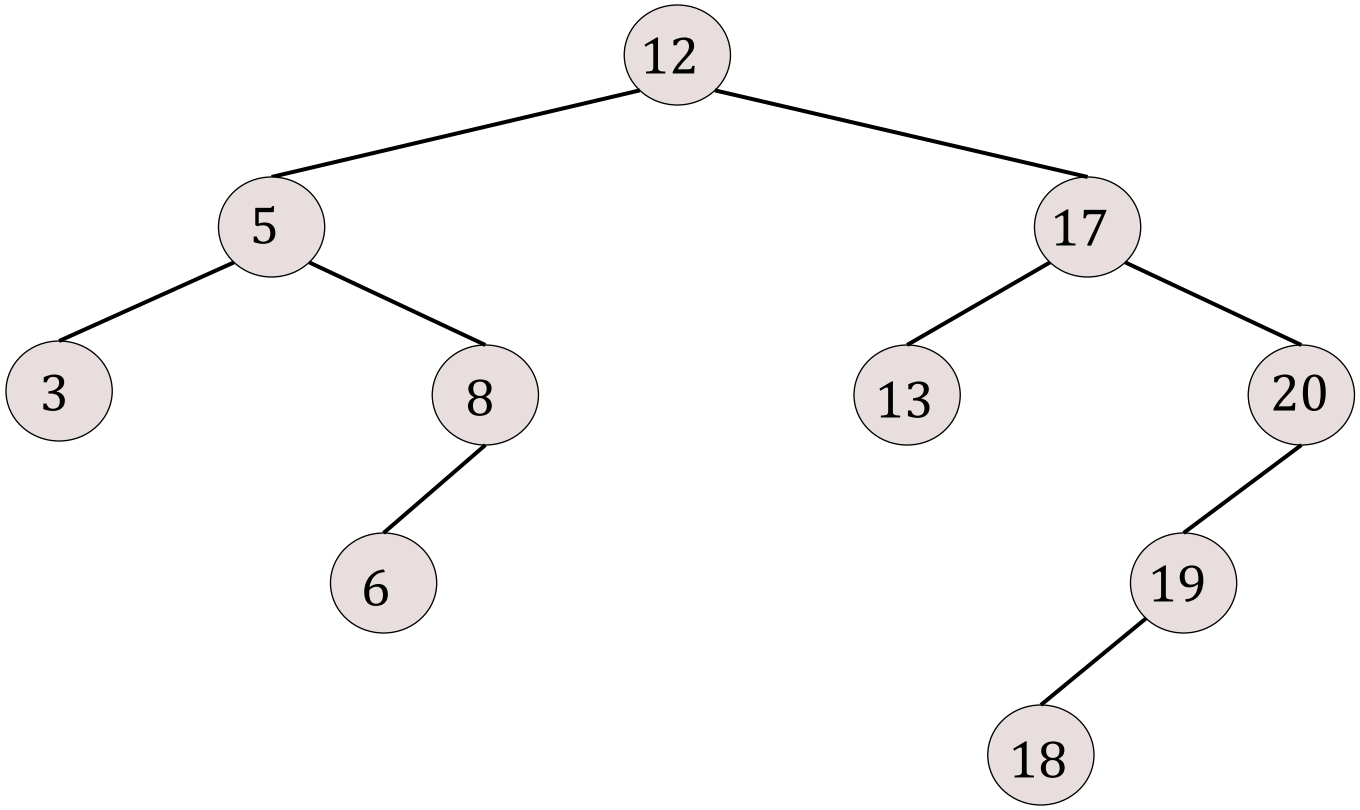



- After $Insert(13, D), Insert(18, D)$
- Time complexity =?

# Quiz

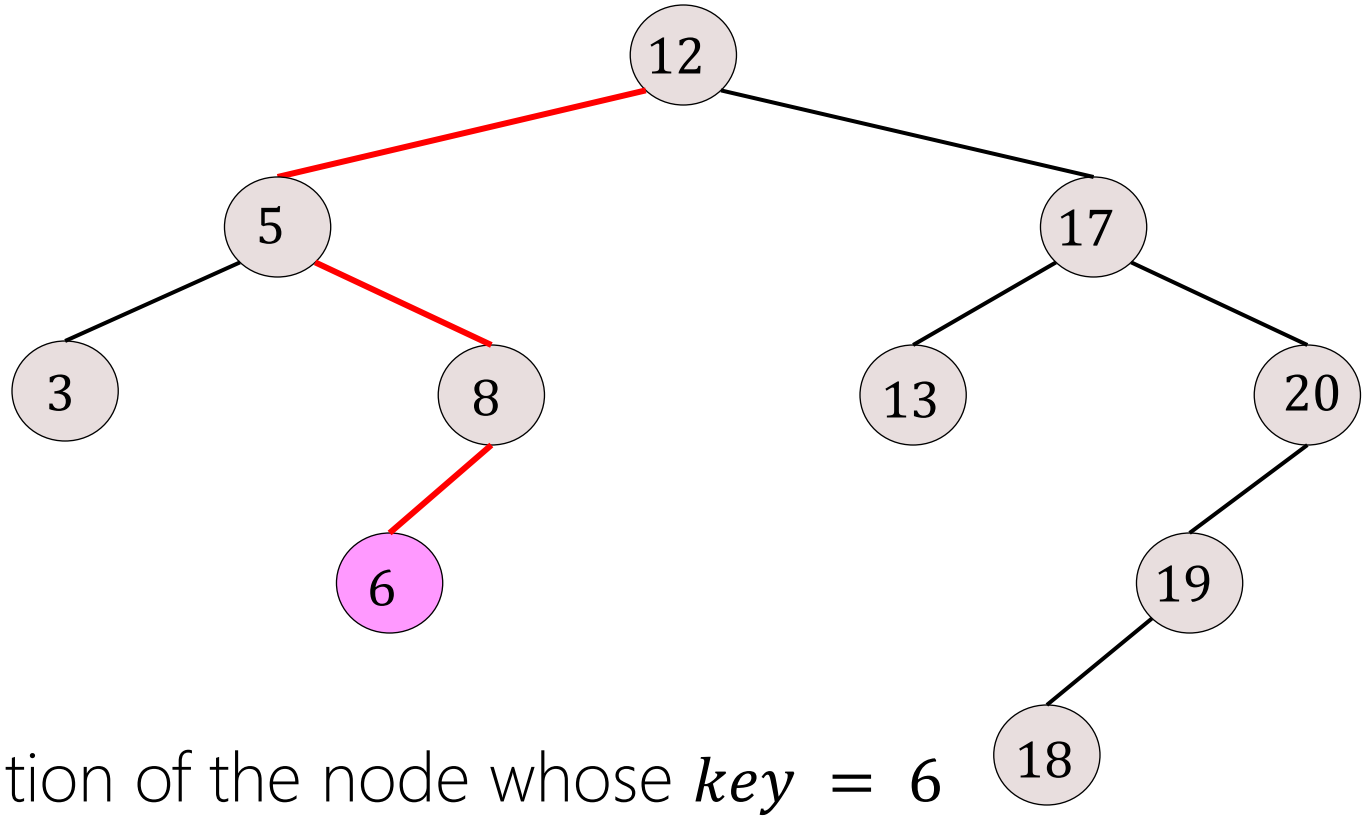
- Two BSTs contain exactly the same set of keys.
- Is it possible that one has height $O(\log n)$, the other has height $O(n)$?

# BST: $Delete(x, D)$ – From a Leaf Node



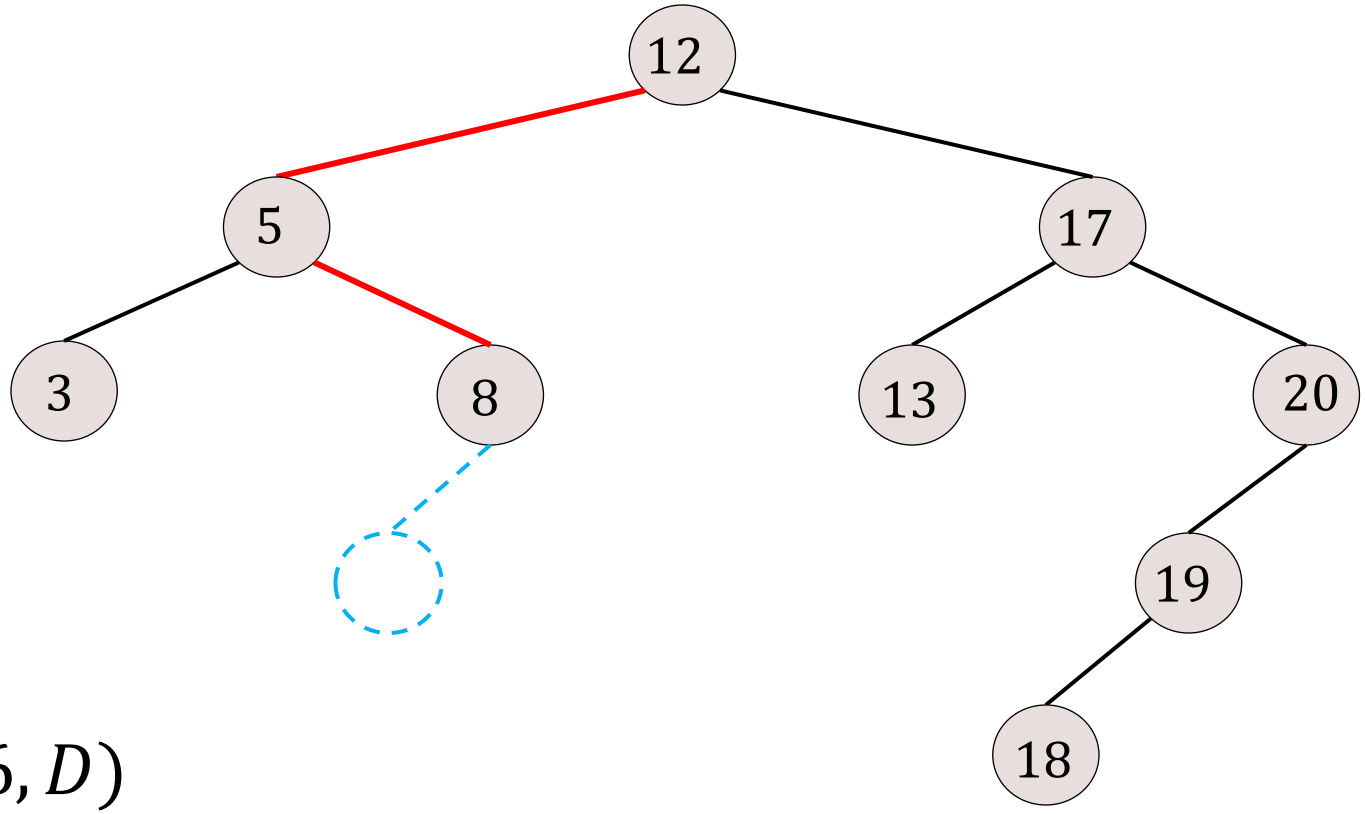

- $Delete(6, D)$

# BST: $Delete(x, D)$ – From a Leaf Node



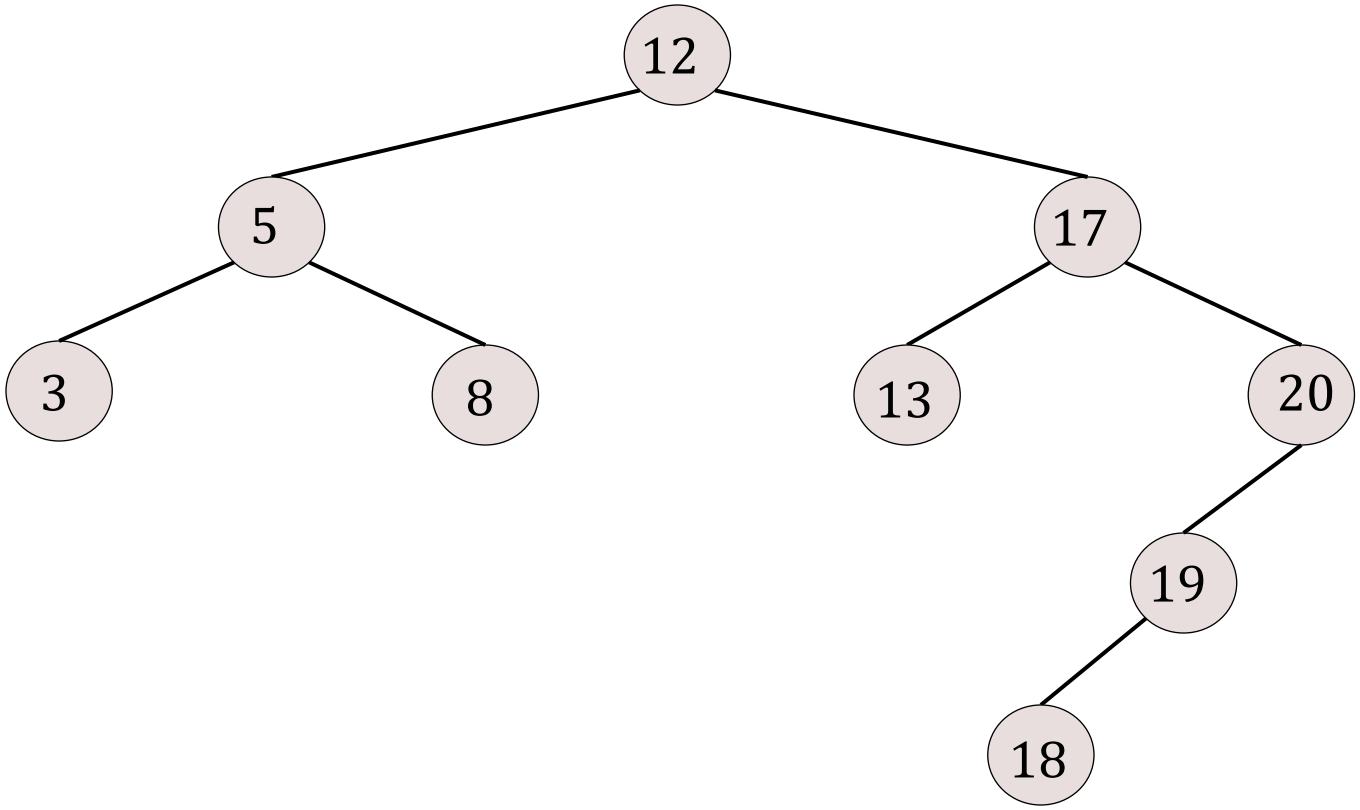

- $Delete(6, D)$
  - Find the location of the node whose $key = 6$
- Easy case: no children

# BST: $Delete(x, D)$ – From a Leaf Node



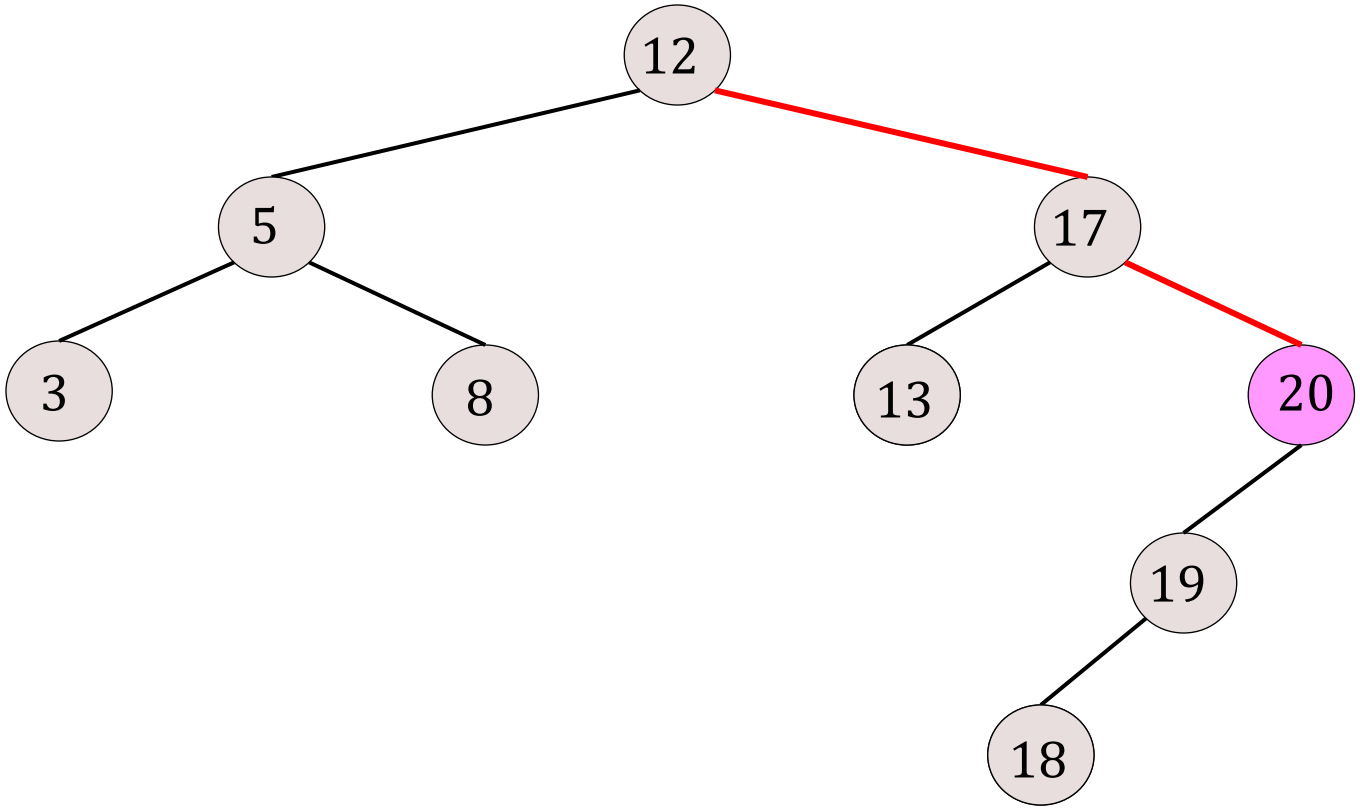

- After $Delete(6, D)$

# BST: $Delete(x, D)$ – From a Degree 1 Node



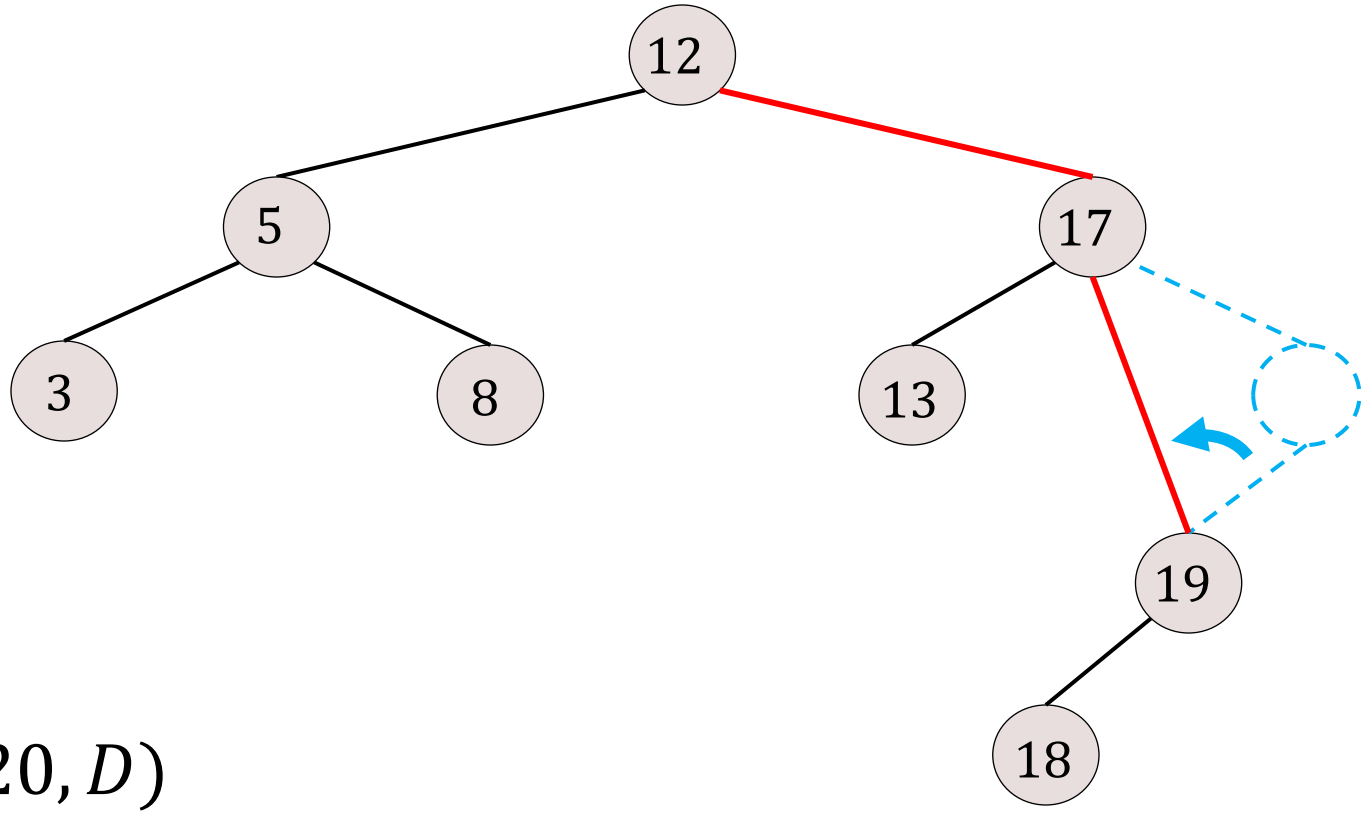

- $Delete(20, D)$

# BST: $Delete(x, D)$ – From a Degree 1 Node



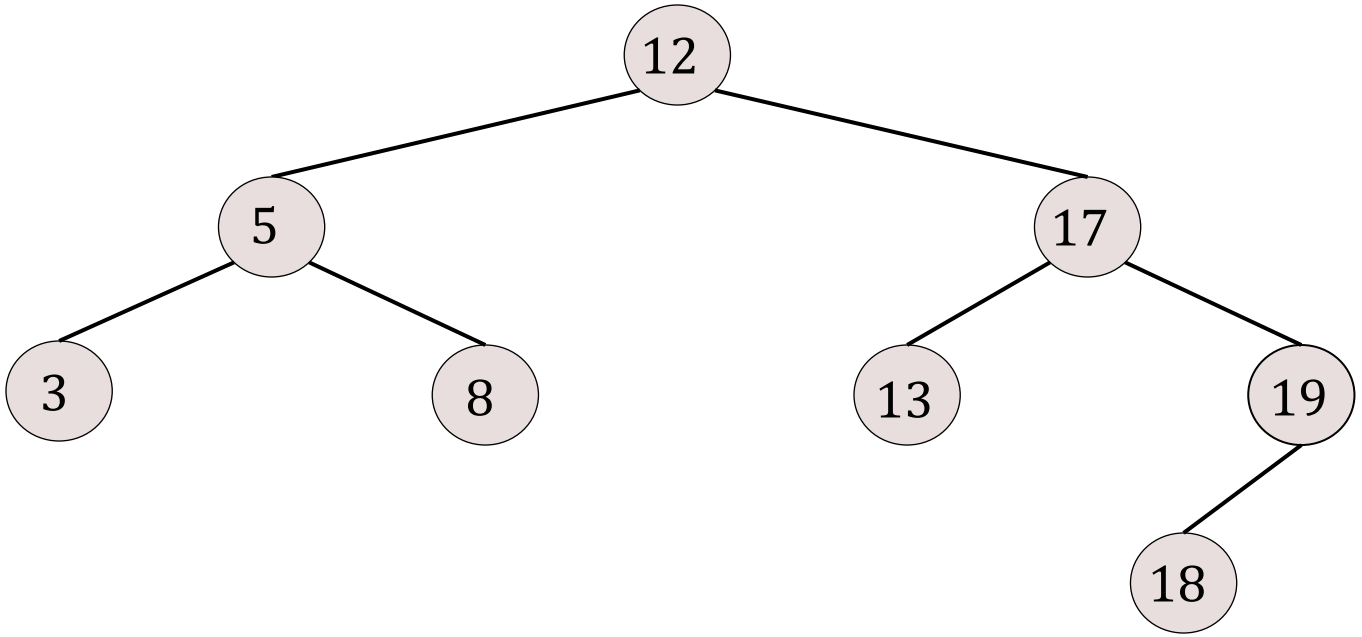

- $Delete(20, D)$
  - Find the location of the node whose $key = 20$
- Almost as easy as the leaf case: only one child

# BST: $Delete(x, D)$ – From a Degree 1 Node



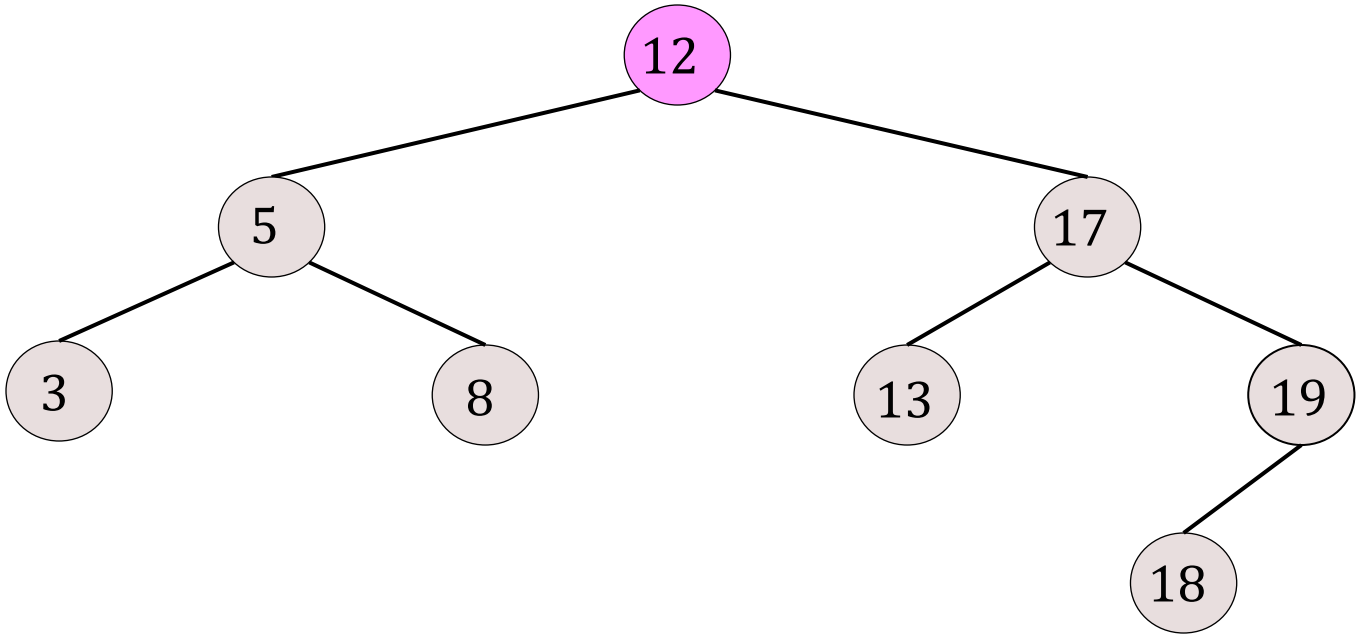

- After $Delete(20, D)$

# BST: $Delete(x, D)$ – From a Deg. 2 Node (1)



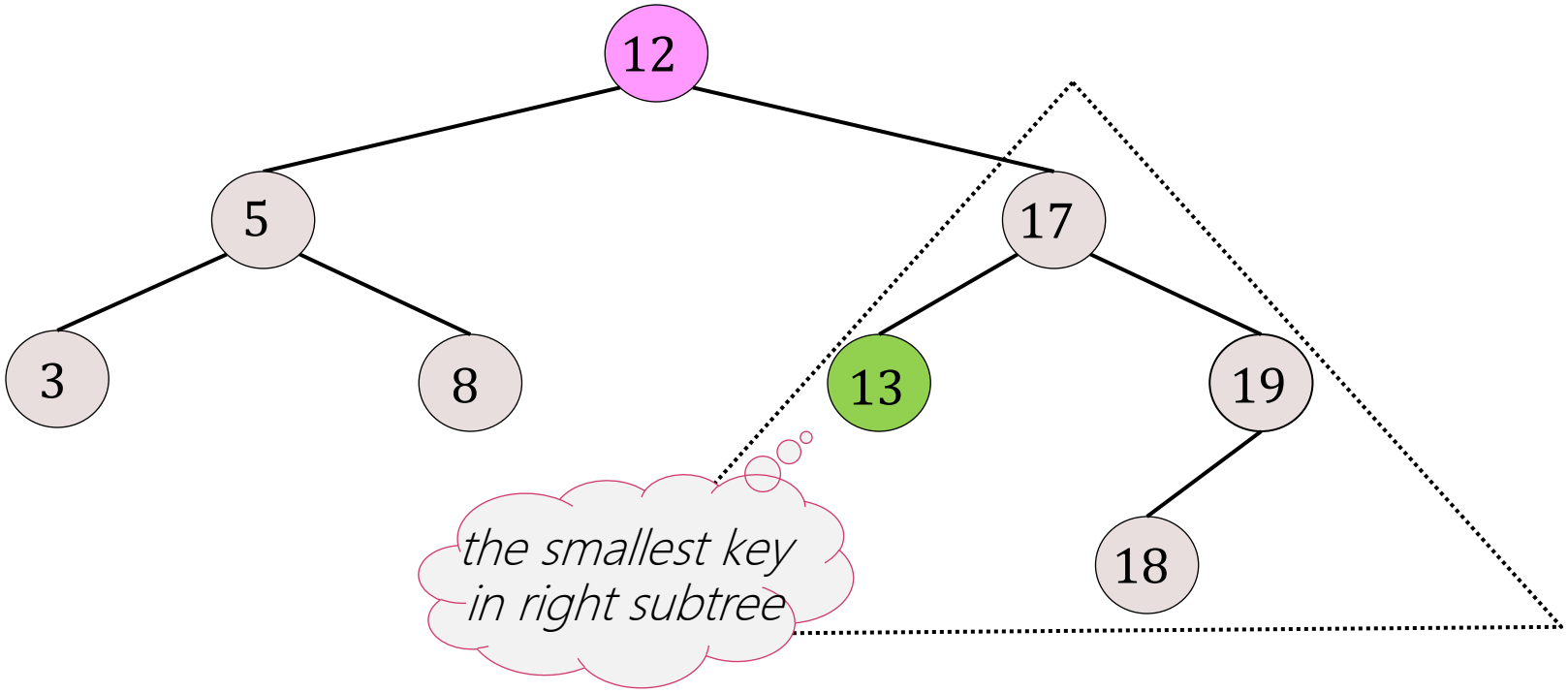

- $Delete(12, D)$

# BST: $Delete(x, D)$ – From a Deg. 2 Node (1)



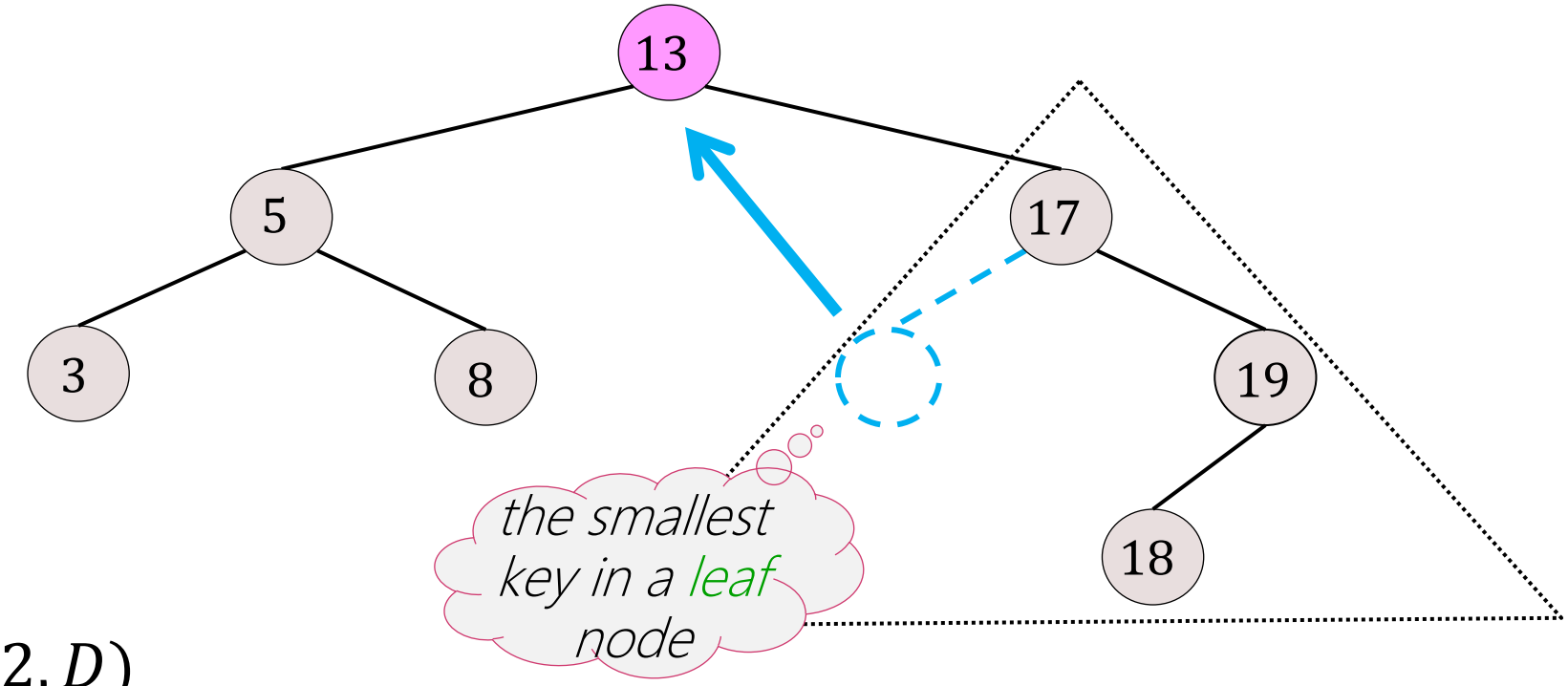

- $Delete(12, D)$
  - Identify the location of the node whose $key = 12$
- Hard case: two children

# BST: $Delete(x, D)$ – From a Deg. 2 Node (1)



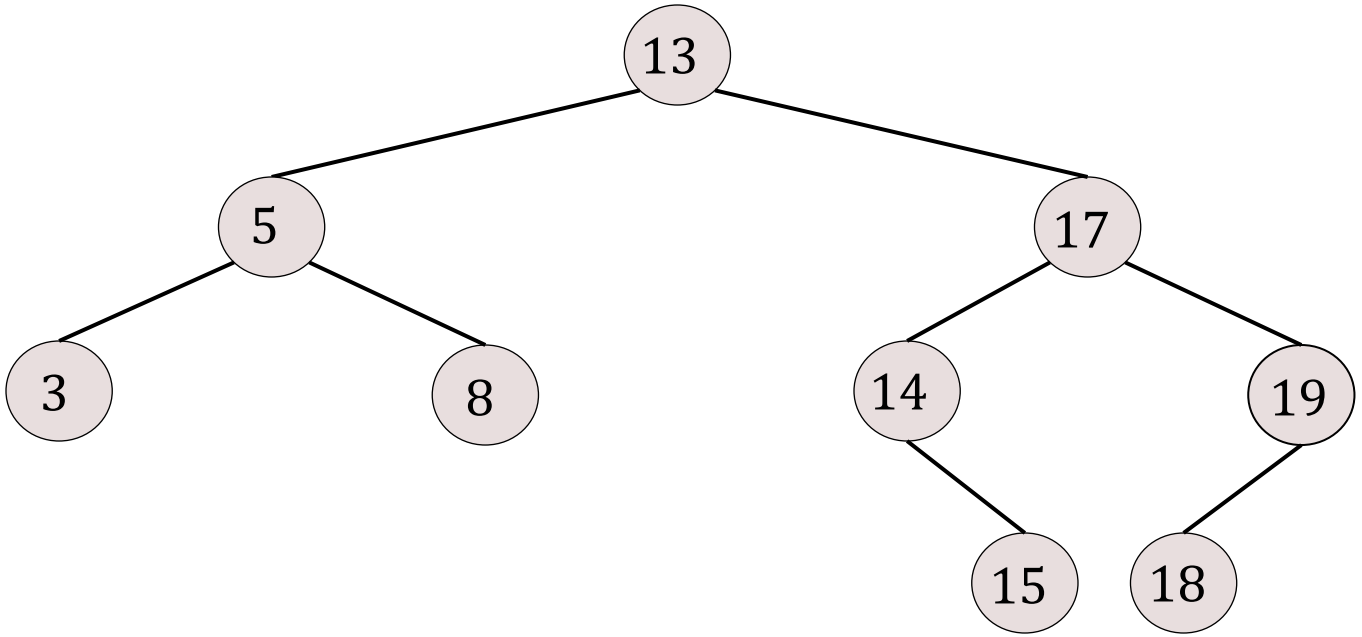

- $Delete(12, D)$
  - Identify the location of the node whose $key = 12$
  - Replace it with the smallest key in its right subtree (or the largest key in its left subtree)

# BST: $Delete(x, D)$ – From a Deg. 2 Node (1)



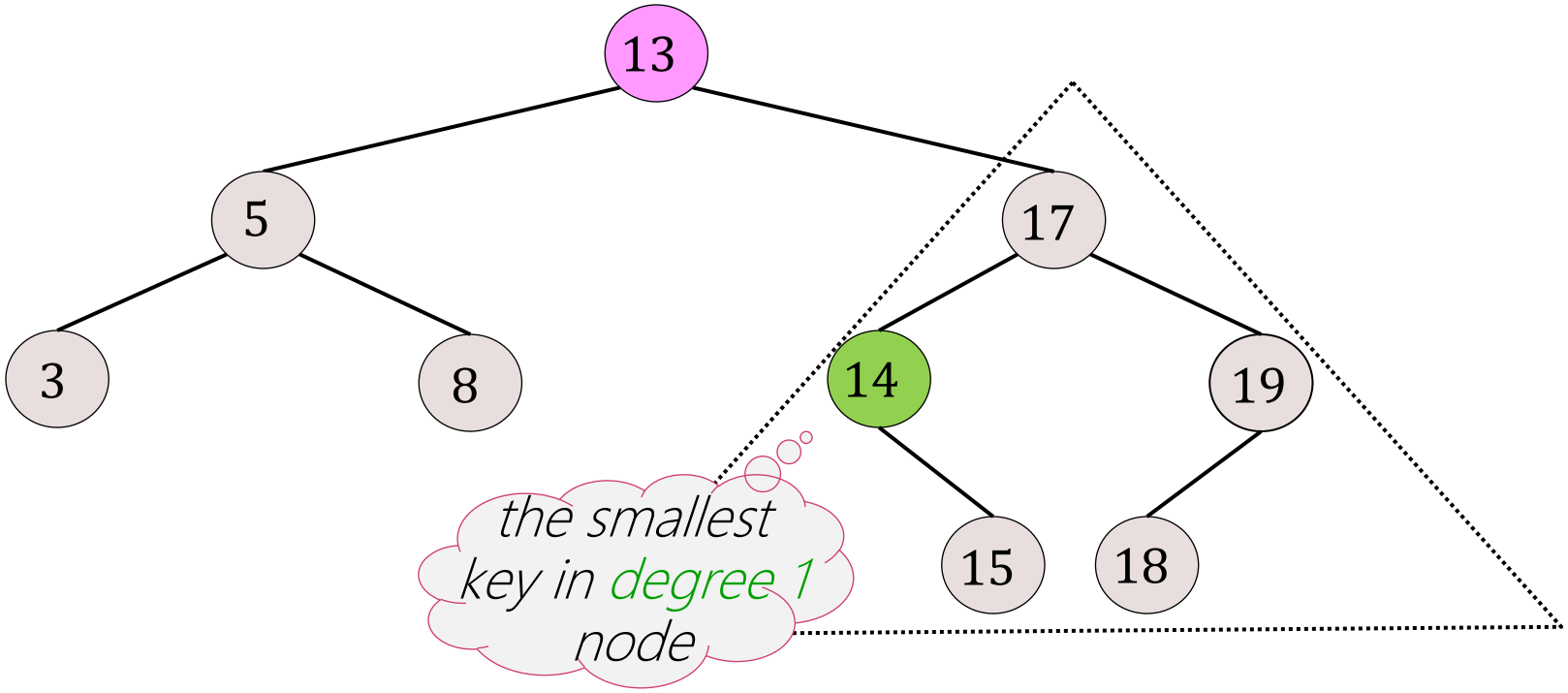

- After $Delete(12, D)$
- Note:
  - The smallest key in the right subtree must be in a leaf (like in this example) or degree 1 node (like in the next example)

# BST: $Delete(x, D)$ – From a Deg. 2 Node (2)



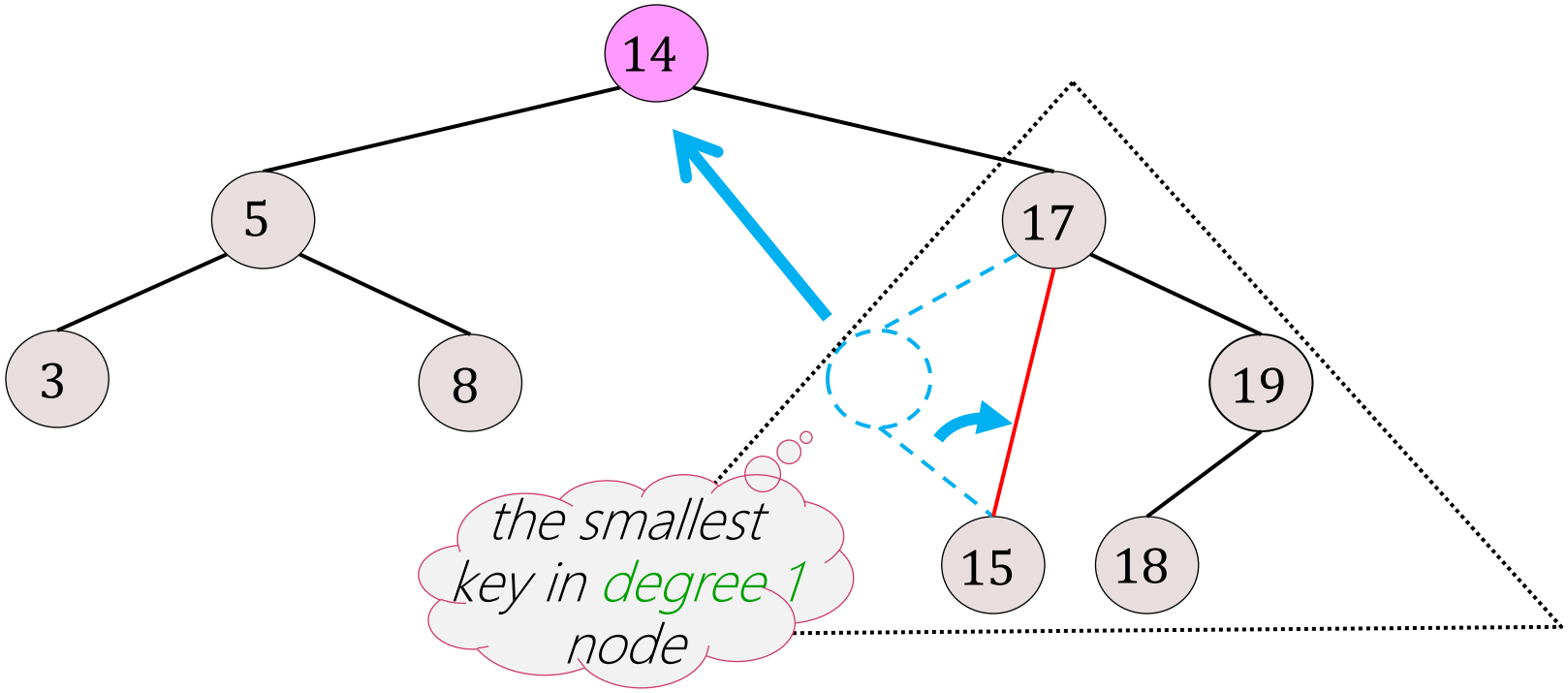

- After $Insert(14, D)$ & $Insert(15, D)$
- Then $Delete(13, D)$

# BST: $Delete(x, D)$ – From a Deg. 2 Node (2)




- $Delete(13, D)$
  - Here, the smallest key is in a degree 1 node

# BST: $Delete(x, D)$ – From a Deg. 2 Node (2)




- After $Delete(13, D)$
- Time complexity = ? $O(height)$

# Quiz

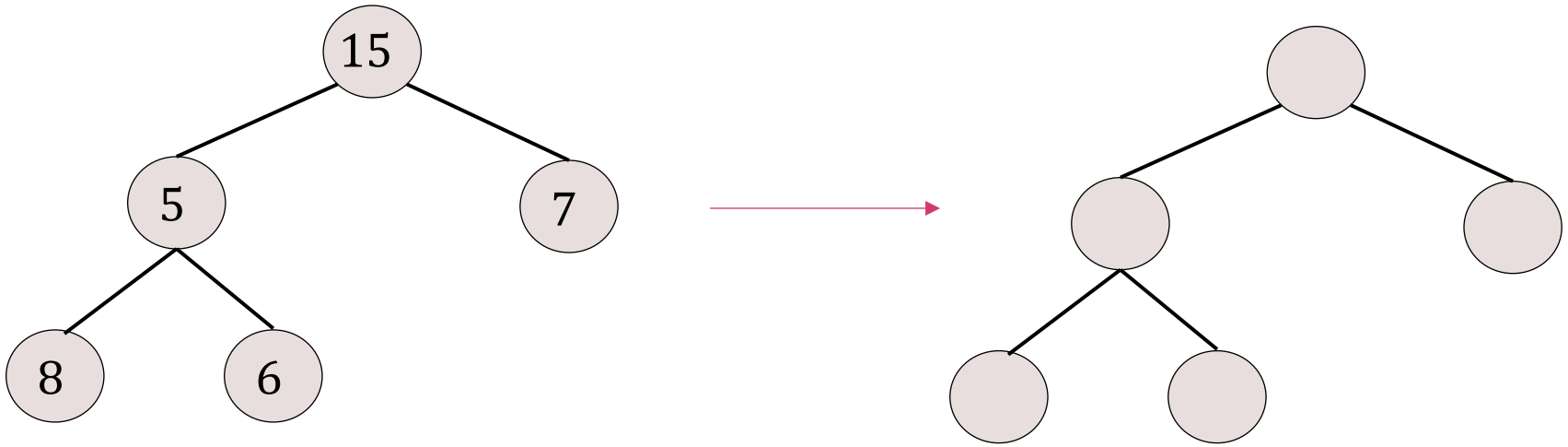
- Suppose you delete a node with two children in a BST by replacing it with the smallest key in its right subtree
- Is it ever necessary to search minimum again after the deletion to restore the BST property? Why or why not?

# Quiz

- When deleting a node with two children by replacing it with the smallest key in its right subtree, why must that replacement node have at most one child?

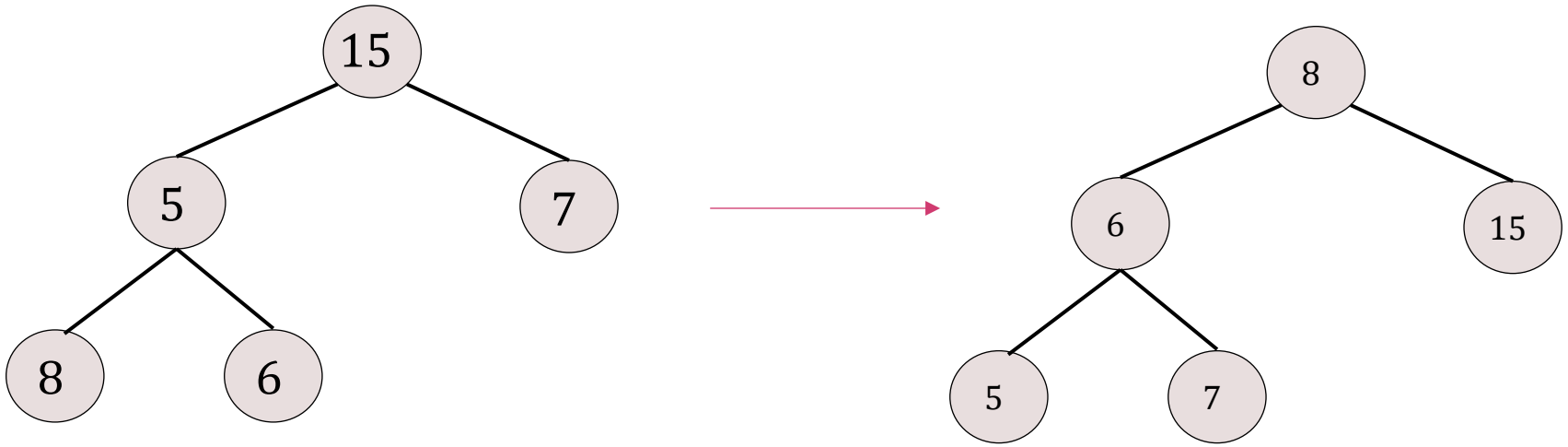
# Convert a Binary Tree into a Binary Search Tree?

- Keep the structure same, but change the values only



# Convert a Binary Tree into a Binary Search Tree?

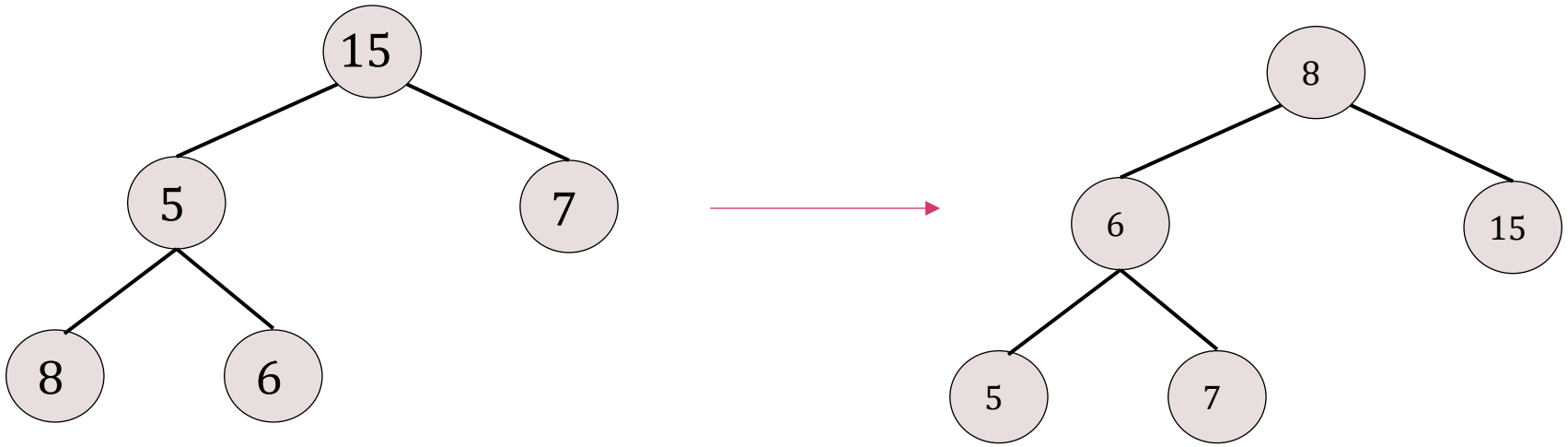
- Keep the structure same, but change the values only




1. Inorder traversal
2. Sort the traversal list
3. Copy the sorted list during another traversal

# Quiz: Convert a Binary Tree into a Binary Search Tree?

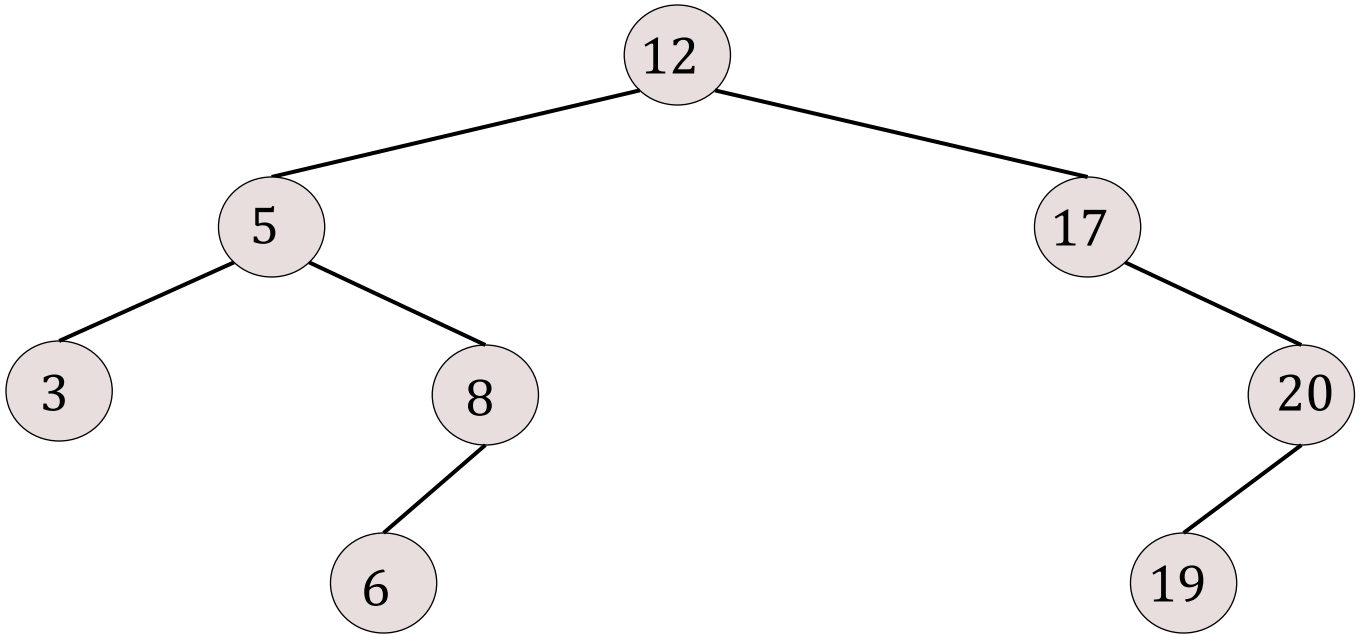
- Keep the structure same, but change the values only




1. Inorder traversal
2. Sort the traversal list
3. Copy the sorted list during another traversal

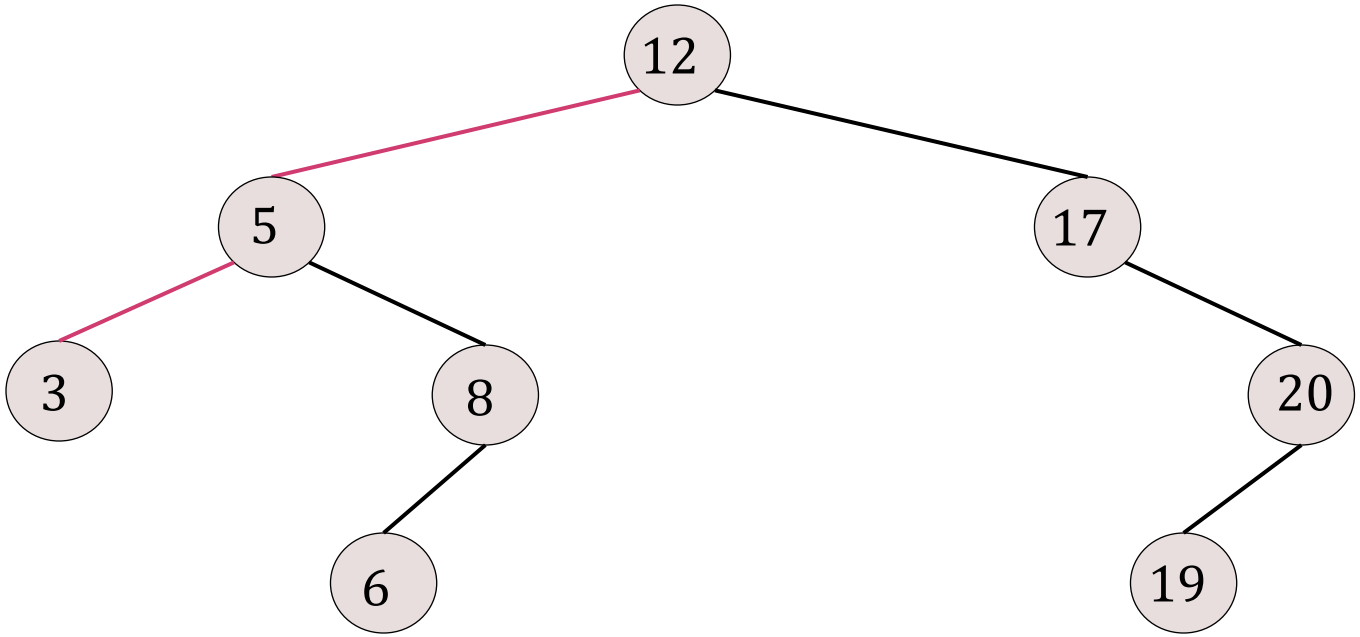
Time complexity?

# BST: $SearchMinimum(D)$




- Find the node with minimum value

# BST: $SearchMinimum(D)$




- Find the node with minimum value
  - Traverse the node from root to left

# Quiz

- Is it possible to support SearchMinimum(D) in O(1) time in a BST without modifying the tree structure?

# Quiz

- A system manages a dynamic set of integer keys and must support the following operations:
  - Insert keys at arbitrary times
  - Search for a given key
  - Remove an arbitrary key
  - Output all keys in ascending order at any time
- Why BST is appropriate for this system?
- What about Priority Queue (heap)?

# Quiz

- In a standard BST, all keys are assumed to be distinct.
- Suppose duplicate keys are allowed in a BST.
- What problems or ambiguities can arise in the design or implementation of the BST?

# Summary

- Binary Search Tree (BST)
- BST Property
- Inorder Traversal
- Search / Insert / Delete
- Tree Height
- Balanced vs Skewed
- Deletion Cases
- Time Complexity